

Diffusion Posterior Sampling for non Exponential Noise

Alexandre Lagier

ENS de Lyon, Computer Science Department

November 2025

Abstract

Inverse problems in imaging and signal processing require recovering clean signals from degraded or incomplete observations. Classical approaches rely on handcrafted priors that fail to capture the complexity of natural data distributions, while recent deep learning methods typically require retraining for each specific inverse problem. We present Diffusion Posterior Sampling (DPS), a method that leverages pre-trained unconditional diffusion models to solve general noisy inverse problems without any model retraining or modification. We run extensive experiments with DPS on real-world datasets to explore its limits. We also refine theoretical justifications of DPS and extend its ability. In the original paper, DPS only handles Gaussian and Poisson noise, however we show that it can also handle Cauchy noise.

1 Introduction

Noisy inverse problems Noisy inverse problems constitute a fundamental class of tasks in computer vision, signal processing, and computational imaging, where the objective is to reconstruct an unknown signal from indirect, incomplete, or degraded observations. These problems arise across numerous domains, including image deblurring, super-resolution, and inpainting. The inherent challenge lies in the ill-posed nature of these problems and the fact that multiple clear images can give the same measurement.

Classical approaches to inverse problems [1] have relied on handcrafted regularizers and optimization-based methods that encode assumptions about natural signal statistics. Although such methods have demonstrated success in specific applications, they often fail to capture complex real-world data. Generative modeling and especially diffusion models come as a bridge between deep learning and classical approaches to inverse problems.

Diffusion Models Diffusion models operate by reversing a noising process through iterative denoising steps. This formulation provides a natural connection to inverse problems, where the measurement is structured with noise. The posterior distribution of the clean signal given noisy observations encodes both the likelihood of the measurements and the learned prior from the diffusion model. However, sampling from this posterior distribution using pre-trained diffusion models presents significant technical challenges, as the standard sampling procedures do not naturally incorporate measurement constraints.

Previous methods [2] used projection onto the measurement space to get sharper results. Although efficient, these methods add more constraints on the transformation, which restricts their use in real-world data.

Diffusion Posterior Sampling In this paper, we present the work of [3] which introduces Diffusion Posterior Sampling (DPS), a diffusion model used to solve inverse problems using pre-trained unconditional diffusion models without requiring any model retraining or modification. The key contribution of **DPS** is an efficient likelihood gradient approximation that enables consistent sampling while preserving the learned prior.

The proposed method operates in a training-free manner, requiring only the forward measurement operator, a noise sampler, and a pre-trained diffusion model. This design ensures broad applicability across diverse inverse problems without the need for problem-specific adaptations. Furthermore, we will show that DPS can handle different noises without having to retrain a different model.

Contribution Our contributions can be summarized as follows: (1) We identified some theoretical holes in the original paper and filled those holes. (2) We derived a new likelihood gradient approximation in the case of Cauchy noise. (3) We performed extensive experiments on standard benchmark datasets such as CelebA and CIFAR10.

First, we recall the principle and theoretical setup of diffusion models and noisy inverse problem. Then, we present the DPS method and its details. Afterwards, we describe the experiment we conducted. Finally, we present an extension of DPS handling Cauchy noise.

2 Preliminaries

2.1 Generative modelling as a denoising process

Diffusion model field is a growing part of generative modeling which is built upon Brownian (or Wiener) motion modellisation and Stochastic Differential Equation (SDE). An SDE can be seen as a noisy differential equation :

$$d\mathbf{x}_t = f(\mathbf{x}_t, t)dt + g(\mathbf{x}_t, t)d\mathbf{w}, \quad (1)$$

where $d\mathbf{w}$ is known as white noise¹ and $g(\mathbf{x}, t)$ represents the deviation of the noise. This kind of differential equation can be seen as a **noising process** in which we start with clean data $\mathbf{x}_0 \sim p_{data}$ and we continuously add a certain amount of noise. In Diffusion, the noising process is bounded in $[0, T]$ where $\mathbf{x}_0$ is a clean image and $\mathbf{x}_T \sim \mathcal{N}(0, \mathbf{I})$. In particular, Diffusion models use a specific SDE known as the Ornstein-Uhlenbeck process :

$$d\mathbf{x}_t = \mu_t \mathbf{x}_t dt + \sigma_t d\mathbf{w}, \quad (2)$$

where $\mu, \sigma \in \mathbb{R}$. More specifically, we use $\mu_t = \beta_t/2$ and $\sigma_t = \sqrt{\beta_t}$ where β_t is an increasing function.

Using Anderson's reversing Theorem [4], we can deduce the following **denoising process**,

$$d\mathbf{x}_t = \left[-\frac{\beta_t}{2} \mathbf{x}_t - \beta_t \nabla_{\mathbf{x}_t} \log p(\mathbf{x}_t) \right] + \sqrt{\beta_t} d\bar{\mathbf{w}}, \quad (3)$$

where $\nabla_{\mathbf{x}_t} \log p(\mathbf{x}_t)$ is the score of the distribution $\mathbf{x}_t$. This makes the denoising process a priori untractable because we cannot compute the score function only by sampling from p_{data} .

2.2 Sampling from a denoising process

As for ordinary differential equation, we can approximate SDE's by using an Euler-Maruyama discretization. For (2) and (3), we discretize $[0, T]$ into N bins and define $\mathbf{x}_t := x_{tT/N}$ and $\beta_t := \beta_{tT/N}$. Regarding [5], we also define $\alpha_t := 1 - \beta_t$ and $\bar{\alpha}_t := \prod_{\tau \leq t} \alpha_\tau$. The noising process can be represented by

$$\mathbf{x}_t = \sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \mathbf{z}, \quad \mathbf{z} \sim \mathcal{N}(0, \mathbf{I}), \quad (4)$$

so that, with Tweedie's formula, we can represent the denoising process by computing the posterior mean at time t ,

$$\hat{\mathbf{x}}_0 := \mathbb{E}[\mathbf{x}_0 | \mathbf{x}_t] = \frac{1}{\sqrt{\bar{\alpha}_t}} \mathbf{x}_0 + \frac{1 - \bar{\alpha}_t}{\sqrt{\bar{\alpha}_t}} \nabla_{\mathbf{x}_t} \log p(\mathbf{x}_t). \quad (5)$$

One can also show that $\nabla_{\mathbf{x}_t} \log p(\mathbf{x}_t)$ is equivalent to a denoiser $\mathbb{E}[\mathbf{z} | \mathbf{x}_t]$. Indeed, [3] choose to approximate the score term by using a denoiser $\varepsilon_\theta(\mathbf{x}_t, t)$ yielding :

$$\hat{\mathbf{x}}_0 \approx \frac{1}{\sqrt{\bar{\alpha}_t}} \mathbf{x}_0 + \sqrt{\frac{1 - \bar{\alpha}_t}{\bar{\alpha}_t}} \varepsilon_\theta(\mathbf{x}_t, t). \quad (6)$$

This method leads to a tractable method, where, once the denoiser ε_θ is learned, we can iteratively denoise a sample $\mathbf{x}_T \sim \mathcal{N}(0, \mathbf{I})$ in order to generate a new sample of p_{data} .

2.3 Inverse noisy problem

Stepping back from diffusion models, we recall Inverse Noisy Problems. Inverse Noisy Problems constitute a fundamental class of problems that arise from numerous scientific fields. In these problems, one seeks to find the original data $\mathbf{x}$ from a measurement $\mathbf{y}$. The measurement is modeled from $\mathbf{x}$ by a two step transformation. First, $\mathbf{x}$ suffers a deterministic transformation $\mathcal{A}(\mathbf{x})$ where $\mathcal{A}$ can be linear or non-linear (e.g a gaussian blur or

¹when integrated between t and $t + \delta t$ it acts like a centered normal distribution with deviation $\sqrt{\delta t}$

non-linear blur). Then, the transformed data $\mathcal{A}(\mathbf{x})$ suffers a non deterministic transformation where random noise is added to $\mathcal{A}(\mathbf{x})$. Formally, this leads to the following relation :

$$\mathbf{y} = \mathcal{A}(\mathbf{x}) + \mathbf{n}, \quad \mathbf{n}, \mathbf{y} \in \mathbb{R}^n, \mathbf{x} \in \mathbb{R}^d. \quad (7)$$

From this arise several questions : How can we find $\mathbf{x}$ back knowing that $\mathbf{y}$ is the result of random process ? How can we even process when $\mathcal{A}$ is many-to-one (ill-posed problem) ? These challenges are further compounded when the noise distribution is unknown or when we lack complete knowledge of the forward operator $\mathcal{A}$. Classical approaches rely on regularization techniques and prior assumptions about $\mathbf{x}$, but these methods often struggle to capture the complex structure of high-dimensional data. As we will see, diffusion models offer a powerful framework for addressing these inverse problems by learning data priors directly from examples, enabling principled posterior sampling even in severely ill-posed settings.

3 Diffusion Posterior Sampling

3.1 Noisy Inverse Problem solved with diffusion models

In this settings, we have a classical noisy inverse problem where we observe $\mathbf{y}$ from $\mathbf{x}$ knowing the transformation $\mathcal{A}$ and the distribution of $\mathbf{n}$. **DPS** is a conditional diffusion model framework used to sample clean original data using noisy processed data. As for usual diffusion models, its generative process is defined as a reverse noising process where it samples noise and generate a clean image. Its formulation is a classical diffusion process in which instead of sampling from the distribution of $\mathbf{x}_t$ it samples from $\mathbf{x}_t|\mathbf{y}$. Formally, the denoising process can be defined as follows,

$$d\mathbf{x}_t = \left[-\frac{\beta_t}{2} \mathbf{x}_t - \beta_t \nabla_{\mathbf{x}_t} \log p(\mathbf{x}_t|\mathbf{y}) \right] dt + \sqrt{\beta_t} d\bar{\mathbf{w}}, \quad (8)$$

where $\bar{\mathbf{w}}$ is a reverse Brownian process. Here, the score term is a priori intractable which makes naive sampling impossible. However, using Bayes law, we have the following score decomposition :

$$\nabla_{\mathbf{x}_t} \log p(\mathbf{x}_t|\mathbf{y}) = \nabla_{\mathbf{x}_t} \log p(\mathbf{x}_t) + \nabla_{\mathbf{x}_t} \log p(\mathbf{y}|\mathbf{x}_t). \quad (9)$$

As we have seen in Section (2), $\nabla_{\mathbf{x}_t} \log p(\mathbf{x}_t)$ can be approximated by a learned denoiser $\varepsilon_\theta(\mathbf{x}_t, t)$. However, the second score term is still untractable. In order to leverage this untractability, **DPS** uses the posterior mean in order to approximate the conditional score $p(\mathbf{y}|\mathbf{x}_t) \approx p(\mathbf{y}|\hat{\mathbf{x}}_0)$. In DPS, this approximation has been detailed and they have shown a bound on the approximation error for the case of Gaussian noise (Theorem 1 of [3]). We also provide a similar bound for the approximation error in the case of Poisson noise in Appendix A (More details and justification of this approximation can be found in [3]).

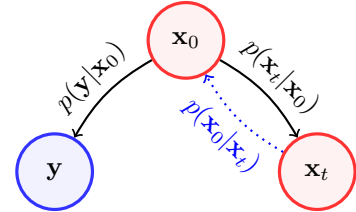


Figure 1: Probabilistic graph. Black solid line: tractable, blue dotted line: intractable in general.

3.2 Approximation of the likelihood

From this last approximation, we present the different ways **DPS** approximates the likelihood $p(\mathbf{y}|\hat{\mathbf{x}}_0)$. The expression of this likelihood will depend on the choice of $\mathcal{A}$, but more importantly on the distribution of the noise $\mathbf{n}$. In all the cases, we will consider i.i.d assumptions for all pixels and all channels.

Gaussian noise. In the case of Gaussian noise, the likelihood has the following form,

$$p(\mathbf{y}|\hat{\mathbf{x}}_0) = \frac{1}{(2\pi\sigma^2)^{n/2}} \exp \left(-\frac{\|\mathbf{y} - \mathcal{A}(\hat{\mathbf{x}}_0)\|_2^2}{2\sigma^2} \right). \quad (10)$$

Differentiating with respect to the diffusion step $\mathbf{x}_t$, we obtain the following log-likelihood gradient,

$$\nabla_{\mathbf{x}_t} \log p(\mathbf{y}|\hat{\mathbf{x}}_0) = -\frac{1}{2\sigma^2} \nabla_{\mathbf{x}_t} \|\mathbf{y} - \mathcal{A}(\hat{\mathbf{x}}_0)\|_2^2. \quad (11)$$

Now, the conditional score in (9) is now tractable, as we can approximate it as follows,

$$\nabla_{\mathbf{x}_t} \log p(\mathbf{x}_t|\mathbf{y}) \approx \varepsilon_\theta(\mathbf{x}_t, t) - \frac{\zeta_t}{2\sigma^2} \nabla_{\mathbf{x}_t} \|\mathbf{y} - \mathcal{A}(\hat{\mathbf{x}}_0)\|_2^2, \quad (12)$$

where ζ_t is the step size of the gradient update. As [3] noticed and as we will show, ζ_t should be chosen carefully to ensure the convergence of the diffusion process.

Poisson noise. In the case of Poisson noise, the likelihood has the following form,

$$p(\mathbf{y}|\hat{\mathbf{x}}_0) = \prod_{i=1}^n \frac{[\mathcal{A}(\hat{\mathbf{x}}_0)]_i^{\mathbf{y}_i} \exp(-[\mathcal{A}(\hat{\mathbf{x}}_0)]_i)}{\mathbf{y}_i!}. \quad (13)$$

However if we compute the log-gradient of this likelihood, the diffusion process will diverge because the gradient would be too unstable. This instability would mainly come from the log term $\nabla_{\mathbf{x}_t} \log[\mathcal{A}(\hat{\mathbf{x}}_0)]_i^{\mathbf{y}_i}$. Instead, [3] suggests to approximate the Poisson distribution by a Gaussian distribution² :

$$p(\mathbf{y}|\hat{\mathbf{x}}_0) \approx \prod_{i=1}^n \frac{1}{\sqrt{2\pi\mathbf{y}_i}} \exp\left(-\frac{(\mathbf{y}_i - [\mathcal{A}(\hat{\mathbf{x}}_0)]_i)^2}{2\mathbf{y}_i^2}\right). \quad (14)$$

This leads to a similar gradient step as for the Gaussian case. We have that :

$$\nabla_{\mathbf{x}_t} \log p(\mathbf{x}_t|\mathbf{y}) \approx \varepsilon_\theta(\mathbf{x}_t, t) - \frac{\zeta_t}{2\sigma^2} \nabla_{\mathbf{x}_t} \|\mathbf{y} - \mathcal{A}(\hat{\mathbf{x}}_0)\|_\Lambda^2, \quad (15)$$

where $\Lambda = \text{diag}((\mathbf{y}_i)_{i \leq n})$.

Cauchy noise. In the case of Cauchy noise, the likelihood has the following form,

$$\begin{aligned} p(\mathbf{y}|\hat{\mathbf{x}}_0) &= \prod_{i=1}^n \frac{1}{\pi\gamma \left(1 + \left(\frac{\mathbf{y}_i - [\mathcal{A}(\hat{\mathbf{x}}_0)]_i}{\gamma}\right)^2\right)} \\ &= (\pi\gamma)^{-n} \prod_{i=1}^n \left(1 + \left(\frac{\mathbf{y}_i - [\mathcal{A}(\hat{\mathbf{x}}_0)]_i}{\gamma}\right)^2\right)^{-1}. \end{aligned}$$

Afterwards, we approximate the log-likelihood in order to make its gradient easily computable :

$$\begin{aligned} \nabla_{\mathbf{x}_t} \log p(\mathbf{y}|\hat{\mathbf{x}}_0) &= - \sum_{i=1}^n \nabla_{\mathbf{x}_t} \log \left(1 + \left(\frac{\mathbf{y}_i - [\mathcal{A}(\hat{\mathbf{x}}_0)]_i}{\gamma}\right)^2\right), \\ &\approx - \sum_{i=1}^n \nabla_{\mathbf{x}_t} \left(\frac{\mathbf{y}_i - [\mathcal{A}(\hat{\mathbf{x}}_0)]_i}{\gamma}\right)^2, \\ &\approx - \frac{1}{\gamma^2} \nabla_{\mathbf{x}_t} \|\mathbf{y} - \mathcal{A}(\hat{\mathbf{x}}_0)\|_2^2, \end{aligned}$$

using $\log(1+x) \approx x$ when x is small. It yields a similar gradient step as the Gaussian noise :

$$\nabla_{\mathbf{x}_t} \log p(\mathbf{x}_t|\mathbf{y}) \approx \varepsilon_\theta(\mathbf{x}_t, t) - \frac{\zeta_t}{\gamma^2} \nabla_{\mathbf{x}_t} \|\mathbf{y} - \mathcal{A}(\hat{\mathbf{x}}_0)\|_2^2, \quad (16)$$

As we can see, in all of the 3 gradients steps, we do not impose any constraint on $\mathcal{A}$. As [3] highlights, the strength of **DPS** comes from its adaptability where the problems can be both linear or non-linear as long as the operator $\mathcal{A}$ is stable enough (Details in Appendix A).

²in high noise regime, this approximation is very accurate.

Algorithm 1 DPS - Gaussian

Require: $N, \mathbf{y}, \{\zeta_t\}_{t=1}^N, \{\tilde{\sigma}_t\}_{t=1}^N$

```
1:  $\mathbf{x}_N \sim \mathcal{N}(0, \mathbf{I})$ 
2: for  $t = N - 1$  to 0 do
3:    $\varepsilon \leftarrow \varepsilon_\theta(\mathbf{x}_t, t)$ 
4:    $\hat{\mathbf{x}}_0 \leftarrow \frac{1}{\sqrt{\bar{\alpha}_t}}(\mathbf{x}_t + \sqrt{1 - \bar{\alpha}_t}\varepsilon)$ 
5:    $\mathbf{z} \sim \mathcal{N}(0, \mathbf{I})$ 
6:    $\mathbf{x}'_{t-1} \leftarrow \frac{\sqrt{\bar{\alpha}_t(1-\bar{\alpha}_{t-1})}}{1-\bar{\alpha}_t}\mathbf{x}_t + \frac{\sqrt{\bar{\alpha}_{t-1}\beta_t}}{1-\bar{\alpha}_t}\hat{\mathbf{x}}_0 + \tilde{\sigma}_t\mathbf{z}$ 
7:    $\mathbf{x}_{t-1} \leftarrow \mathbf{x}'_{t-1} - \zeta_t \nabla_{\mathbf{x}_t} \|\mathbf{y} - \mathcal{A}(\hat{\mathbf{x}}_0)\|_2^2$ 
8: end for
9: return  $\hat{\mathbf{x}}_0$ 
```

Algorithm 2 DPS - Poisson

Require: $N, \mathbf{y}, \{\zeta_t\}_{t=1}^N, \{\tilde{\sigma}_t\}_{t=1}^N$

```
1:  $\mathbf{x}_N \sim \mathcal{N}(0, \mathbf{I})$ 
2: for  $t = N - 1$  to 0 do
3:    $\varepsilon \leftarrow \varepsilon_\theta(\mathbf{x}_t, t)$ 
4:    $\hat{\mathbf{x}}_0 \leftarrow \frac{1}{\sqrt{\bar{\alpha}_t}}(\mathbf{x}_t + \sqrt{1 - \bar{\alpha}_t}\varepsilon)$ 
5:    $\mathbf{z} \sim \mathcal{N}(0, \mathbf{I})$ 
6:    $\mathbf{x}'_{t-1} \leftarrow \frac{\sqrt{\bar{\alpha}_t(1-\bar{\alpha}_{t-1})}}{1-\bar{\alpha}_t}\mathbf{x}_t + \frac{\sqrt{\bar{\alpha}_{t-1}\beta_t}}{1-\bar{\alpha}_t}\hat{\mathbf{x}}_0 + \tilde{\sigma}_t\mathbf{z}$ 
7:    $\mathbf{x}_{t-1} \leftarrow \mathbf{x}'_{t-1} - \zeta_t \nabla_{\mathbf{x}_t} \|\mathbf{y} - \mathcal{A}(\hat{\mathbf{x}}_0)\|_\Lambda^2$ 
8: end for
9: return  $\hat{\mathbf{x}}_0$ 
```

3.3 General method

We sum up the method of Diffusion Posterior Sampling by exhibiting the algorithms 1,2 shown in [3]. The general method remains close to an unconditional diffusion, where we only add a gradient step between the posterior mean and the measurement. Notice that when the measurement noise changes, only the gradient step needs to be modified. Compared to the original **DPS**, we have shown that Cauchy noise can also be handled with a similar method.

4 Implementation Details

Here are some comments that we can make on our implementation of Diffusion Posterior Sampling method.

Diffusion implementation.

- For the denoiser scheduler, we used the DDPM scheduler presented in [5]
- We used the DiffUNet model for ε_θ with the pretrained weight for the FFHQ dataset from [3]
- Compared to the original DPS algorithm, we found it beneficial to return $\mathbf{x}_0$ and not $\hat{\mathbf{x}}_0$.
- For $\tilde{\sigma}_t$, we used the learned variance given by the DiffUNet model. However, fixing $\tilde{\sigma}_t = \sqrt{\beta_t}$ as in DDPM yields similar results.

Measurement implementation.

- We implemented the following transformations : Super-Resolution, Inpainting, Gaussian blur, Motion blur (linear), Phase-retrieval (non-linear). The Gaussian blur and Motion blur code was imported from two different repositories: [bkse](#), [motionblur](#).
- We implemented the following noises : Gaussian, Poisson, Binomial, Gamma, Cauchy. However, experiments have shown that Gamma and Binomial noises are hard to handle with no appropriate likelihood approximation.
- To simulate Poisson, Binomial, and Gamma noises, we assumed that each measurement pixel is a source of photons where the number of photons is proportional to the discrete pixel value in $\llbracket 0, 255 \rrbracket$. Therefore, to add noise to $\mathcal{A}(\mathbf{x}_0)$, we map $[-1, 1]$ to $\llbracket 0, 255 \rrbracket$ then add the noise and finally map it back to $[-1, 1]$. A similar method is mentioned in DPS [3] for Poisson noise.

Evaluation implementation.

- We implemented the following metrics : FID [6], LPIPS [7], and PSNR.
- FID was implemented using the InceptionV3 model which maps each image to a learned embedding $\in \mathbb{R}^{2048}$ and then we compute the FID using those embeddings.
- LPIPS was implemented using the python package [lpips](#). In particular, we use the VGG model.

5 Experiments

5.1 Experiments setup

We perform our experiment on two datasets that represent different objects : CelebA 256×256 [8] and CIFAR10 32×32 [9]. The pretrained diffusion model used for CelebA was taken from [3] which was trained on FFHQ 256×256 . The pretrained diffusion model used for CIFAR10 was taken from [10] which was trained on ImageNet 256×256 . We have tested DPS method using the following operators : (i) For super-resolution, we used a $\times 4$ linear operator; (ii) For inpainting, we used a 128×128 box mask for CelebA and 16×16 for CIFAR10 (iii) For Gaussian blur, we used a kernel size of 61 for CelebA and a kernel size of 10 for CIFAR10, both with intensity 3.0 (iv) For Motion blur, we used a kernel size of 61 for CelebA and a kernel size of 10 for CIFAR10, both with intensity 0.5 (v) For Phase retrieval, we used the Fourier magnitude extracted from the Discrete Fourier Transform (DFT) with oversampling 2.0. When unspecified, all Gaussian noise is added with $\sigma = 0.05$, all Poisson noise with $\lambda = 1.0$ and all Cauchy noise with $\gamma = 1.0$. Code of the experiments is available at <https://github.com/a-lagier/DPS-Cauchy>.

5.2 Resources

All experiments were conducted on a Nvidia GA102 [RTX 3090 Ti] GPU with 24GB of memory. All experiments were conducted using Python 3.8 and Torch. Due to the limited time, we tested DPS on CelebA with 250 images sampled from the validation set and CIFAR10 with 1k images sampled from the validation set.

5.3 Baselines of DPS

First, we conducted baseline experiments on CelebA dataset with both Gaussian and Poisson noise (see Table 1). In [3], DPS shows great performance on the FFHQ dataset. The objective is to verify whether these performances are maintained on a similar dataset. Indeed, to reconstruct images on the CelebA dataset we used the same unconditional denoiser that was used for the FFHQ dataset in [3].

Task	LPIPS (CelebA) ↓	LPIPS (FFHQ) ↓	Task	LPIPS (CelebA) ↓
Super-resolution	0.244 ± 0.002	0.214	Super-resolution	0.172 ± 0.001
Inpainting	0.248 ± 0.001	0.168	Inpainting	0.178 ± 0.003
Gaussian Blur	0.289 ± 0.001	0.257	Gaussian Blur	0.209 ± 0.000
Motion Blur	0.277 ± 0.001	0.242	Motion Blur	0.165 ± 0.001

Table 1: Quantitative evaluation (LPIPS) of DPS with Gaussian $\sigma = 0.05$ (left) and Poisson $\lambda = 1.0$ (right) noise on CelebA dataset with linear transforms. **LPIPS (CelebA)** results were taken from [3].

For the Gaussian noise, we observe comparable results to those on the FFHQ dataset. However, results on FFHQ remain provably higher for all the operators. This could be explained by the fact that the denoiser, which is the same for the two datasets experiments, was trained on the FFHQ dataset.

For the Poisson noise, we observe that DPS is able to find suitable clean image from the noisy measurement. This empirically validates the approximation used for the gradient step. The fact that we observe better reconstruction for Poisson noise than for the Gaussian noise could be explained by the difference of their noise processing. With Poisson noise, a pixel value $\in \llbracket 0, 255 \rrbracket$ deviates from $\lambda = 1.0$ whereas, with Gaussian noise, it deviates from $\frac{256}{2} \times \sigma = 6.4$.

We also conducted experiments on CIFAR10 32×32 (see Appendix B).

5.4 Non-linear operator

One of the breakthrough of DPS is its ability to solve non-linear noisy inverse problems the same way it operates with linear problems. Here, we tested the ability of DPS to reconstruct an image knowing a noisy DFT of the original image. In Table 2, the experiment shows that our reconstruction of Phase-Retrieval fails. Original DPS finds a LPIPS value of 0.399 for FFHQ. We choose

Task	LPIPS ↓	PSNR ↑
Phase Retrieval	0.676	10.93
±	0.002	1.55

Table 2: Quantitative evaluation (LPIPS, PSNR) phase retrieval with Gaussian noise on CelebA

not to go any further as it appears that our implementation of Phase-Retrieval is incorrect (compared to the one implemented in [3]).

5.5 Hyperparameters analysis

In order to test DPS stability, we choose to run hyperparameter analysis experiment where we vary the value of one hyperparameter. Two key hyperparameters are the number of time steps of the diffusion process T and the step size ζ of the gradient update.

Time steps. As for any denoising process, choosing T accordingly is crucial to obtain both sharp and accurate denoised images. Because the pretrained models that we use are generally trained with $T = 1000$, when we make T vary, we need to rescale the time value t so that it starts at $t = 1000$ and ends at $t = 0$.

Timesteps T	LPIPS $\downarrow$	PSNR (dB) $\uparrow$
$T = 100$	0.795 ± 0.001	10.04 ± 0.31
$T = 200$	0.759 ± 0.000	10.83 ± 0.72
$T = 500$	0.515 ± 0.001	17.40 ± 0.57
$T = 1000$	0.289 ± 0.001	25.33 ± 0.37
$T = 2000$	0.722 ± 0.009	5.13 ± 23.73

Table 3: Effect of number of timesteps on reconstruction quality. Experiment on CelebA 256×256 with Motion blur and Gaussian noise.

In Table 3, we observe that, for medium values of T , the more steps we add the more accurate the reconstruction would be. However, when we set $T = 2000$, the accuracy crashes. Indeed, the time rescaling forces the denoising process to repeat 2 times each steps. Due to the stochastic nature of the step, it causes the denoising to fail because it tries to denoise the same image in two different directions.

Step size. Another important hyperparameter in DPS is the step size ζ of the gradient step. As pointed out by [3], its choice is crucial in order to reconstruct the original image : a small ζ could lead to a generation of a different image and a large ζ could amplify the measurement noise in the reconstructed image.

Step size ζ	LPIPS $\downarrow$	PSNR (dB) $\uparrow$
$\zeta = 0.05$	0.562 ± 0.000	15.03 ± 0.20
$\zeta = 0.1$	0.503 ± 0.000	17.19 ± 0.44
$\zeta = 0.2$	0.455 ± 0.001	19.08 ± 0.21
$\zeta = 0.5$	0.377 ± 0.000	22.05 ± 0.50
$\zeta = 1.0$	0.244 ± 0.002	28.01 ± 0.38
$\zeta = 2.0$	0.270 ± 0.000	27.56 ± 0.20
$\zeta = 5.0$	0.259 ± 0.000	27.80 ± 0.97

Table 4: Effect of gradient step size on super-resolution with Gaussian noise (CelebA). Experiment on CelebA 256×256 with SR and Gaussian noise.

In Table 4, we observe that in low step size regime, we naturally obtain poor reconstruction performance. However, interestingly, in high step size regime, we tend to observe more concentrated performances. This could be explained by the stability of the operator (here Super-Resolution). Further work could be to confirm this hypothesis by conducting the same experiment with Phase-Retrieval, a drastically less stable operator.

5.6 Extension

Finally, we conducted a baseline experiment with a new type of noise : Cauchy noise. The main property of this noise is to be heavy-tailed. For high values of noise, adding noise would act as masking a fraction of the image.

Task	LPIPS ↓	PSNR (dB) ↑
Super-resolution	0.333 ± 0.000	24.49 ± 0.41
Inpainting	0.276 ± 0.001	21.78 ± 0.26
Gaussian Blur	0.307 ± 0.001	25.39 ± 0.84
Motion Blur	0.322 ± 0.000	24.46 ± 0.01

Table 5: Cauchy noise baseline on CelebA dataset.

Because of this, we expect similar yet lower performances than with Gaussian noise due to their distributions similarity.

In Table 5, as expected we observe lower performance for all the operators. However, the performances are good enough to empirically validate the approximation done in the gradient step. DPS is able to denoise the Cauchy in the measured image. Due to the lack of literature on this type of noise, further work would be to experience the effect of Cauchy noise on other inverse problem solvers.

6 Conclusion

In this paper, we presented Diffusion Posterior Sampling method. We have shown its link with both noisy inverse problems and diffusion models. Throughout careful approximation, we proved that it could be extended as other noise distributions could be considered. We also re-implemented the original method and ran experiments on other datasets. From these experiments, we have established that DPS is an accurate method with broad applicability across various inverse problems. However, we also noticed that its inner algorithm could lack flexibility : changing the number of iterations greatly impacts its performances.

References

- [1] Singanallur V. Venkatakrishnan, Charles A. Bouman, and Brendt Wohlberg. Plug-and-play priors for model based reconstruction. In *2013 IEEE Global Conference on Signal and Information Processing*, pages 945–948, 2013.
- [2] Hyungjin Chung, Byeongsu Sim, Dohoon Ryu, and Jong Chul Ye. Improving diffusion models for inverse problems using manifold constraints. In Sanmi Koyejo, S. Mohamed, A. Agarwal, Danielle Belgrave, K. Cho, and A. Oh, editors, *Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems 2022, NeurIPS 2022, New Orleans, LA, USA, November 28 - December 9, 2022*, 2022.
- [3] Hyungjin Chung, Jeongsol Kim, Michael T Mccann, Marc L Klasky, and Jong Chul Ye. Diffusion posterior sampling for general noisy inverse problems. In *International Conference on Learning Representations*, 2023.
- [4] Brian D.O. Anderson. Reverse-time diffusion equation models. *Stochastic Processes and their Applications*, 12(3):313–326, 1982.
- [5] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. In H. Larochelle, M. Ranzato, R. Hadsell, M.F. Balcan, and H. Lin, editors, *Advances in Neural Information Processing Systems*, volume 33, pages 6840–6851. Curran Associates, Inc., 2020.
- [6] Martin Heusel, Hubert Ramsauer, Thomas Unterthiner, Bernhard Nessler, and Sepp Hochreiter. Gans trained by a two time-scale update rule converge to a local nash equilibrium. In *Advances in Neural Information Processing Systems*, volume 30, pages 6626–6637, 2017.
- [7] Richard Zhang, Phillip Isola, Alexei A Efros, Eli Shechtman, and Oliver Wang. The unreasonable effectiveness of deep features as a perceptual metric. In *CVPR*, 2018.
- [8] Ziwei Liu, Ping Luo, Xiaogang Wang, and Xiaoou Tang. Deep learning face attributes in the wild. In *Proceedings of International Conference on Computer Vision (ICCV)*, December 2015.

- [9] Alex Krizhevsky. Learning multiple layers of features from tiny images. 2009.
- [10] Prafulla Dhariwal and Alexander Nichol. Diffusion models beat gans on image synthesis. In M. Ranzato, A. Beygelzimer, Y. Dauphin, P.S. Liang, and J. Wortman Vaughan, editors, *Advances in Neural Information Processing Systems*, volume 34, pages 8780–8794. Curran Associates, Inc., 2021.

A Proof of Poisson approximation bound

We show that the approximation error can also be bounded in the case of Poisson noise. First, we show a slightly more general result.

Theorem A.1. For the current measurement model described in [3], for $\mathbf{n} \sim X$ with f_X the pdf of X , we have,

$$p(\mathbf{y}|\mathbf{x}_t) \approx p(\mathbf{y}|\hat{\mathbf{x}}_0), \quad (17)$$

where the approximation error can be quantified with the Jensen gap, which is upperbounded by,

$$\mathcal{I} \leq L_{f_X} \|\nabla_{\mathbf{x}} \mathcal{A}(\mathbf{x})\| m_1, \quad (18)$$

where L_{f_X} is the Lipschitz norm of f_X , $\|\nabla_{\mathbf{x}} \mathcal{A}(\mathbf{x})\| := \max_{\mathbf{x}} \|\nabla_{\mathbf{x}} \mathcal{A}(\mathbf{x})\|$ and $m_1 := \int \|\mathbf{x}_0 - \hat{\mathbf{x}}_0\| p(\mathbf{x}_0|\mathbf{x}_t) d\mathbf{x}_0$.

Proof.

$$p(\mathbf{y}|\mathbf{x}_t) = \int p(\mathbf{y}|\mathbf{x}_0) p(\mathbf{x}_0|\mathbf{x}_t) d\mathbf{x}_0 \quad (19)$$

$$= \mathbb{E}_{\mathbf{x}_0 \sim p(\mathbf{x}_0|\mathbf{x}_t)} [f(\mathbf{x}_0)] \quad (20)$$

Here, $g(\cdot) := f_X(\mathcal{A}(\cdot))$ where $\mathcal{A}$ is the forward operator and $f_X(\mathbf{x})$ is the multivariate normal distribution with mean $\mathbf{y}$ and the covariance $\sigma^2 I$. Therefore, we have

$$J(g, p(\mathbf{x}_0|\mathbf{x}_t)) = |\mathbb{E}[g(\mathbf{x}_0)] - g(\mathbb{E}[\mathbf{x}_0])| = |\mathbb{E}[g(\mathbf{x}_0)] - g(\hat{\mathbf{x}}_0)| \quad (21)$$

$$= |\mathbb{E}[f_X(\mathcal{A}(\mathbf{x}_0))] - f_X(\mathcal{A}(\hat{\mathbf{x}}_0))| \quad (22)$$

$$\leq \int |f_X(\mathcal{A}(\mathbf{x}_0)) - f_X(\mathcal{A}(\hat{\mathbf{x}}_0))| dP(\mathbf{x}_0|\mathbf{x}_t) \quad (23)$$

$$\leq L_{f_X} \int \|\mathcal{A}(\mathbf{x}_0) - \mathcal{A}(\hat{\mathbf{x}}_0)\| dP(\mathbf{x}_0|\mathbf{x}_t) \quad (24)$$

$$\leq L_{f_X} \|\nabla_{\mathbf{x}} \mathcal{A}(\mathbf{x})\| \int \|\mathbf{x}_0 - \hat{\mathbf{x}}_0\| dP(\mathbf{x}_0|\mathbf{x}_t) \quad (25)$$

$$\leq L_{f_X} \|\nabla_{\mathbf{x}} \mathcal{A}(\mathbf{x})\| m_1 \quad (26)$$

where the second inequality can be deduce because f_X is Lipschitz. The other inequalities are deduce as in Theorem 1 of [3]. $\square$

From this, all we have to show is that f_P the pdf of Poisson distribution is Lipschitz.

Proposition A.2. Let $f(\cdot)$ be a univariate Poisson density function with parameter λ . There exists a constant L such that $\forall x, y \in \mathbb{R}$,

$$|f(x) - f(y)| \leq L|x - y|. \quad (27)$$

Proof. As f' is continuous and bounded, we use the mean value theorem to get,

$$\forall x, y \in \mathbb{R}, |f(x) - f(y)| \leq \|f'\|_{\infty} |x - y|. \quad (28)$$

Because $f(x) = \frac{x^\lambda e^{-x}}{\lambda!}$, we have the following relations,

$$f'(x) = f(x) \left(\frac{\lambda}{x} - 1 \right),$$

$$f''(x) = f(x) \left(\frac{k^2 - k}{x^2} - \frac{2k}{x} + 1 \right).$$

By solving a quadratic equation $f''(x) = 0$, we show the existence of $x_{\pm} = k \pm \sqrt{k}$ where f' attain its optimum. We thus have shown that $\|f'\|_{\infty}$ is finite and that f is Lipschitz. $\square$

As a consequence of this two latter results, we have shown that the approximation in the case of Poisson noise is provably bounded.

B Additional experiments

In addition to testing DPS on a similar dataset as FFHQ (namely CelebA), we also conducted a baseline on CIFAR10, a similar dataset as ImageNet. Due to the limited time, we only ran the baseline on two different operators. Here, we observe poor results compared to the results on ImageNet (see [3]). This could be explained by the notable different resolutions of the two datasets.

Task	LPIPS $\downarrow$	PSNR (dB) $\uparrow$
Super-resolution	0.566 ± 0.000	13.68 ± 0.03
Inpainting	0.390 ± 0.001	17.55 ± 0.16
Gaussian Blur	0.531 ± 0.000	13.43 ± 0.01
Motion Blur	0.378 ± 0.000	20.20 ± 0.06

Table 6: Gaussian noise baseline on CIFAR10 dataset.